

Networking Exercises

Bài 1: TCP + Client - Server (Mô hình Echo)

Xây dựng một hệ thống chat bao gồm một Client và một Server dựa trên mô hình Echo sử dụng giao thức TCP. Hệ thống cần đáp ứng các yêu cầu sau:

- Hệ thống phải sử dụng Sockets để giao tiếp và hỗ trợ ít nhất một kết nối hoạt động.
- Server phải nhận được tin nhắn và ngay lập tức trả về chính xác tin nhắn đó cho Client.
- Client phải hiển thị phản hồi nhận được từ Server lên màn hình.

Bài 2: Multi-Thread Client Server

Mở rộng hệ thống chat ở Exe 5 với tính năng đa luồng (multi-threading) để hỗ trợ giao tiếp theo thời gian thực cho nhiều người dùng cùng lúc. Hệ thống cần đáp ứng các yêu cầu sau:

- Server phải xử lý nhiều kết nối Client đồng thời mà không làm chặn (block) luồng thực thi chính khi một Client cụ thể phản hồi chậm.
- Khi một Client gửi tin nhắn, Server phải xử lý và trả về phản hồi tương ứng cho đúng người gửi đó.
- Đảm bảo rằng Client nhận và hiển thị thành công phản hồi của Server trên màn hình.
- Triển khai cơ chế ghi nhật ký (logging) để theo dõi các thông tin cần thiết như trạng thái kết nối, địa chỉ IP của client, các sự kiện giao tiếp, v.v.

Bài 3: REST API

Xây dựng một hệ thống chat theo phòng (room-based) sử dụng kiến trúc REST API phi trạng thái (stateless) dựa trên giao thức HTTP. Hệ thống cần đáp ứng các yêu cầu sau:

- Server phải quản lý các ID Client duy nhất được chỉ định theo thứ tự tham gia và cung cấp các endpoint để tham gia (join), gửi tin nhắn (submit), và liệt kê toàn bộ giao tiếp trong phòng.
- Client phải tham gia vào phòng để nhận ID, gửi tin nhắn được gắn thẻ với ID đó, và định kỳ lấy danh sách tin nhắn để hiển thị.
- Các Client trước tiên phải gửi một yêu cầu đến Server để đăng ký và tham gia phòng trước khi có thể bắt đầu trò chuyện.

- Khi một Client gửi tin nhắn, Server nhận và phát sóng (broadcast) nó vào không gian giao tiếp chung, đảm bảo tất cả các Client khác trong cùng một phòng đều có thể truy cập được tin nhắn đó.

Bài 4: Thiết kế CSDL Quan hệ & Chuẩn hóa (Relational Database Design & Normalization)

Dựa trên hệ thống chat theo phòng ở bài tập REST API (Exe 7), thiết kế một Cơ sở dữ liệu Quan hệ hoàn chỉnh để lưu trữ dữ liệu lâu dài thay vì chỉ lưu trên RAM của Server. Hệ thống cần đáp ứng các yêu cầu sau:

- Thiết kế một Sơ đồ Thực thể - Mối quan hệ (ERD) chứa ít nhất ba thực thể cốt lõi: User (Người dùng), Room (Phòng), và Message (Tin nhắn), xác định rõ các mối quan hệ Một-Nhiều (1-N) hoặc Nhiều-Nhiều (N-M).
- Chuẩn hóa lược đồ cơ sở dữ liệu đạt Chuẩn 3 (3NF) để loại bỏ sự dư thừa dữ liệu và đảm bảo tính toàn vẹn của dữ liệu.
- Viết các tập lệnh SQL (Ngôn ngữ Định nghĩa Dữ liệu - DDL) (**CREATE TABLE**) để tạo các bảng này. Bắt buộc phải áp dụng các ràng buộc cần thiết bao gồm **PRIMARY KEY**, **FOREIGN KEY**, **NOT NULL**, và các giá trị **DEFAULT** (ví dụ: đặt thời gian mặc định của tin nhắn là thời gian hiện tại).

Bài 5: Truy vấn dữ liệu nâng cao với Cơ sở dữ liệu mẫu Sakila

Tải và thiết lập cơ sở dữ liệu mẫu **sakila** chính thức của MySQL (một hệ thống quản lý cửa hàng cho thuê DVD). Viết các câu truy vấn SQL (Ngôn ngữ Thao tác Dữ liệu - DML) nâng cao để giải quyết các yêu cầu phân tích dữ liệu sau:

- Viết một câu truy vấn sử dụng JOIN cơ bản và các hàm Tổng hợp (Aggregation) để tìm ra Top 5 thể loại phim (categories) tạo ra tổng doanh thu cho thuê cao nhất.
- Viết một câu truy vấn sử dụng Lọc phức tạp (mệnh đề **HAVING**) để lấy danh sách các khách hàng đã thuê nhiều hơn 30 bộ phim, hiển thị Tên, Email và Tổng số lượt thuê của họ, sắp xếp theo số lượng thuê giảm dần.
- Viết một câu truy vấn sử dụng Truy vấn con (Subqueries) hoặc Biểu thức Bảng Chung (CTE) để xác định những khách hàng có tổng chi tiêu lớn hơn mức chi tiêu trung bình của toàn bộ khách hàng cộng lại.
- Viết một câu truy vấn sử dụng Window Functions để gán Thứ hạng (Rank) cho các bộ phim trong từng thể loại cụ thể dựa trên thời lượng (length) của

chúng, hiển thị Tên phim, Thể loại, Thời lượng và Thứ hạng tương ứng.

Bài 6: Lập trình mạng với Hệ thống Giám sát Thời tiết Thông minh (TCP vs. UDP)

Xây dựng một hệ thống giao tiếp đa kênh cho Trạm Thời tiết, bao gồm Kênh Điều khiển bằng TCP (đảm bảo độ tin cậy 100% cho các lệnh quan trọng) và Kênh Viễn trắc bằng UDP (ưu tiên tốc độ truyền phát dữ liệu cảm biến theo thời gian thực). Viết các chương trình mạng và áp dụng kỹ thuật xử lý ngoại lệ (Exception Handling) để giải quyết các yêu cầu sau:

- Viết một class `CommandServer` sử dụng Lập trình Socket (TCP) với `ServerSocket` để lắng nghe trên cổng 5000, nhận và phản hồi các lệnh ("START" in ra "System initialized...", "SHUTDOWN" in ra "System shutdown..."). Yêu cầu tích hợp thời gian chờ `setSoTimeout(5000)` để bắt `SocketTimeoutException` nếu không có tương tác, đồng thời xử lý `BindException` và `IOException`.
- Viết một class `CommandClient` sử dụng `Socket` (TCP) để kết nối tới `localhost` cổng 5000, thực hiện gửi tuần tự lệnh "START" và "SHUTDOWN". Yêu cầu sử dụng khối try-catch để bắt `ConnectException` và hiển thị cảnh báo "Error: Remote server is offline!", qua đó kiểm chứng tính tin cậy của TCP khi phía Server chưa khởi động.
- Viết một class `SensorSender` (Trạm thời tiết) sử dụng `DatagramSocket` và `DatagramPacket` (UDP) để gửi chuỗi dữ liệu môi trường (ví dụ: "Temp: 28°C, Humidity: 65%") đến cổng 6000.
- Viết một class `SensorReceiver` (Trạm giám sát) sử dụng `DatagramSocket` (UDP) lắng nghe trên cổng 6000 để nhận các gói tin (packet) liên tục và hiển thị nội dung dữ liệu thời tiết nhận được lên màn hình console.
- Viết một báo cáo phân tích ngắn (Critical Thinking) dựa trên quá trình kiểm thử hệ thống để giải thích: (1) Ngoại lệ nào xảy ra và tại sao khi chạy đồng thời hai chương trình `CommandServer` trên cùng một cổng; (2) Sự khác biệt về bản chất giao thức khiến TCP (`CommandClient`) báo lỗi khi vắng Server, trong khi UDP (`SensorSender`) gửi dữ liệu mà không báo lỗi dù không có Receiver.

MVC, GUI, JavaFX Exercises

Bài 1: SceneBuilder và FXML

Thiết kế một biểu mẫu nhập liệu cho một lĩnh vực mà bạn tự chọn, chẳng hạn như hệ thống đăng ký sinh viên, phần mềm ghi nhận sách thư viện, hoặc trình quản lý kho sản phẩm. Trọng tâm của bài tập là tạo ra một bố cục giao diện người dùng có tổ chức và chuyên nghiệp. Hệ thống cần đáp ứng các yêu cầu sau:

- Xây dựng giao diện bằng công cụ SceneBuilder và lưu bố cục hoàn chỉnh dưới dạng một tệp FXML.
- Kết hợp ít nhất 3 loại Bố cục (Layouts) khác nhau để cấu trúc biểu mẫu một cách hiệu quả (ví dụ: sử dụng VBox làm khung chứa chính, GridPane để căn chỉnh các trường nhập liệu, và HBox cho các nút bấm hành động ở dưới cùng).
- Triển khai đa dạng các Điều khiển giao diện (UI Controls), ví dụ như TextField, CheckBox, Button, v.v.
- Áp dụng CSS để định dạng kiểu dáng trực quan cơ bản cho các thành phần giao diện.

Bài 2: Triển khai mô hình MVC - Event Handling

Sử dụng biểu mẫu nhập liệu đã thiết kế ở Exe 1, triển khai kiến trúc MVC để quản lý logic ứng dụng và các tương tác của người dùng. Hệ thống cần đáp ứng các yêu cầu sau:

- Triển khai logic Xử lý sự kiện (Event Handling) sao cho việc nhấp vào nút “Submit” sẽ kích hoạt Controller đọc toàn bộ các giá trị hiện tại từ biểu mẫu.
- Thực thi quy trình kiểm tra dữ liệu (Validation) để phát hiện các lỗi, chẳng hạn như trường bị bỏ trống hoặc sai định dạng. Cung cấp phản hồi trực quan trên giao diện để thông báo cho người dùng về việc gửi thành công hoặc các lỗi xác thực cụ thể.
- Duy trì sự phân tách vai trò nghiêm ngặt, đảm bảo rằng View chỉ đơn thuần là một bố cục khai báo, trong khi Controller xử lý toàn bộ logic tương tác.

Bài 3: Lưu trữ dữ liệu với Database trong mô hình MVC

Dựa trên dự án MVC đã tích hợp từ Exe 2, bạn có nhiệm vụ triển khai tính năng

lưu trữ dữ liệu để đảm bảo thông tin do người dùng nhập vào có thể được bảo tồn và tải lại giữa các phiên chạy ứng dụng. Hệ thống cần đáp ứng các yêu cầu sau:

- Tích hợp ứng dụng với một Cơ sở dữ liệu (Database) sử dụng JDBC.
- Triển khai tính năng Save (Lưu) trong Controller nhằm kích hoạt Model chèn các dữ liệu biểu mẫu đã được xác thực vào Cơ sở dữ liệu. Hiển thị một cảnh báo hoặc thông báo trực quan để xác nhận bản ghi đã được lưu thành công hay đã xảy ra lỗi cơ sở dữ liệu.
- Triển khai tính năng Load (Tải) để truy xuất các bản ghi đã lưu và tự động điền (populate) dữ liệu từ Cơ sở dữ liệu vào các điều khiển giao diện JavaFX.

Bài 4: Lập trình sự kiện JavaFX nâng cao & Xử lý đa luồng (Concurrency)

Sử dụng biểu mẫu nhập liệu và kiến trúc MVC từ các bài trước, nâng cấp trải nghiệm người dùng (UX) và hiệu năng của ứng dụng bằng cách áp dụng các kỹ thuật xử lý sự kiện nâng cao và đa luồng. Hệ thống cần đáp ứng các yêu cầu sau:

- Triển khai kiểm tra dữ liệu theo thời gian thực (Real-time Validation) bằng cách sử dụng Change Listeners (`textProperty().addListener(...)`). Nếu người dùng nhập sai dữ liệu, lập tức đổi màu viền của TextField sang màu đỏ và hiển thị cảnh báo, sau đó chuyển lại bình thường khi dữ liệu được sửa đúng.
- Cấu hình Sự kiện Bàn phím (Keyboard Events) để cho phép người dùng chuyển con trỏ (focus) sang trường nhập liệu tiếp theo bằng phím "Enter", và tạo một phím tắt toàn cục (ví dụ: Ctrl + S) để kích hoạt quá trình gửi dữ liệu giống hệt như khi bấm nút "Submit/Save".
- Tái cấu trúc (Refactor) các thao tác Lưu (Save) và Tải (Load) dữ liệu để chạy bất đồng bộ trên một luồng nền (ví dụ: sử dụng `javafx.concurrent.Task` hoặc `Thread`) nhằm tránh làm đơ (freeze) luồng giao diện chính.
- Hiển thị một `ProgressIndicator` (vòng xoay tải) trực quan trên giao diện trong khi luồng nền đang chạy, và bắt buộc sử dụng `Platform.runLater(...)` để cập nhật giao diện an toàn với thông báo thành công hoặc thất bại khi tác vụ hoàn thành.

